

LLM Fundamentals

Messages & Roles in LLMs

Study notes for Agentic AI development

Core mental model

System / Developer -> User -> Assistant -> Tool

These messages become part of the context available to the model.

This lesson focuses on how message roles organize instructions, user requests, assistant responses, and external tool results in an LLM application.

What you will learn

- Why LLM applications use message roles
- What user, system/developer, assistant, and tool messages represent
- How tool calling connects an LLM to APIs and databases
- The difference between tool definition, tool call, and tool result
- How messages and tool results contribute to context engineering
- A complete transaction-agent example and failure-handling pattern

1. Why do we need roles?

When you communicate with an LLM through an application or API, you do not necessarily send one giant piece of text. The conversation is organized into messages, and messages have roles.

System / Developer -> User -> Assistant -> Tool

Role	Purpose
System / Developer	How should the model behave?
User	What does the user want or provide?
Assistant	What has the model said or done?
Tool	What did an external tool return?

The application needs a way to distinguish instructions governing behavior from the user request. That is where message roles come in.

```
System / Developer Instructions
|
User
|
Assistant
|
Tool
|
LLM
```

2. User Message

The user message represents what the user asks or provides.

User
Show me my last five transactions.

User
What is our ATM withdrawal policy?

User
Compare these two accounts.

```
User request
|
LLM
|
Understand intent
|
Decide what to do
The user message tells the system what the user wants.
```

3. System / Developer Instructions

These provide instructions controlling how the model or application should behave.

```
You are a banking support agent.  
  
You can help users retrieve account information.  
  
Never initiate a financial transaction without  
explicit user confirmation.  
  
Use customer tools when current account data  
is required.  
  
Keep responses concise.
```

The user may ask:

```
Transfer $500 to John.
```

But the instructions say that explicit confirmation is required. The application might therefore respond:

```
Please confirm that you want to transfer $500 to John.
```

4. Why instructions matter for agents

For a normal chatbot, instructions might be simple:

- Be concise.
- Answer in English.
- Explain technical concepts simply.

For an agent, instructions can become more operational:

```
You are a customer-service agent.  
  
For account balance questions:  
use get_account_balance.  
  
For transaction questions:  
use get_transactions.  
  
Never guess account information.  
  
If a tool fails:  
inform the user instead of inventing data.  
  
For money transfers:  
obtain confirmation before execution.
```

These instructions begin to define agent behavior.

5. Assistant Messages

An assistant message represents model-generated content in the conversation.

User: What is RAG?

Assistant: RAG stands for Retrieval-Augmented Generation...

If the conversation continues, the previous assistant response may be included in the context.

System instructions

+

User: What is RAG?

+

Assistant: RAG stands for...

+

User: Why do we need it?

|

LLM

As conversations grow, conversation history can consume more and more context.

User

Assistant

User

Assistant

User

Assistant

...

6. Tool Messages - Very Important for Agentic AI

Suppose the user asks: "What is my account balance?" The LLM should not invent a balance. Instead, the agent may have a tool such as:

```
get_account_balance(account_id)
```

User

"What's my balance?"

|

LLM

|

"I need account data."

|

Tool call:

get_account_balance(123)

|

Banking API / Database

|

Tool result:

```
{
  "account_id": 123,
  "balance": 25000
}
```

|

LLM

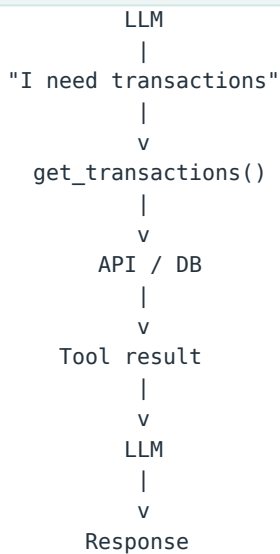
|

"Your account balance is 25,000."

The external result becomes information the model can use.

7. Important distinction: LLM vs Tool

The LLM decides / interprets.
The tool executes an external operation.



Suppose the database returns:

```
{
  "transactions": [
    {"merchant": "Amazon", "amount": 120},
    {"merchant": "Uber", "amount": 30}
  ]
}
```

The LLM can now reason over real tool-provided data rather than guessing.

8. Tool Definition vs Tool Call vs Tool Result

These three concepts are easy to confuse.

Tool Definition

Tells the model what a tool exists for.

```
Tool: get_transactions
Purpose: Retrieve customer transactions.
Inputs: customer_id, limit
```

Tool Call

The model/application decides to invoke it with arguments.

```
get_transactions(
  customer_id=123,
  limit=5
)
```

Tool Result

The actual external system returns data.

```
{
  "customer_id": 123,
  "transactions": [...]
}
```

Tool Definition

|

LLM knows the tool exists

Tool Call

|

Tool is invoked

Tool Result

|

External data returned

LLM

|

Uses result

9. Complete Agent Example

User asks: Show my last five transactions and tell me if anything looks unusual.

Step 1 - Context

```
SYSTEM/DEVELOPER
You are a banking assistant.
Never invent customer financial information.
Use tools for current customer data.
```

```
TOOLS
get_transactions(customer_id, limit)
```

```
USER
Show my last five transactions and tell me
if anything looks unusual.
```

Step 2 - Model decides

The LLM recognizes that it needs transaction information and requests:

```
get_transactions(
  customer_id=123,
  limit=5
)
```

Step 3 - Tool executes

LLM

|

Tool call

|

Python/backend

|

Bank API / Database

The LLM itself is not querying the database directly. Your application/tool does that.

Step 4 - Tool returns

Transaction 1 -> \$20
Transaction 2 -> \$35
Transaction 3 -> \$1,800
Transaction 4 -> \$12
Transaction 5 -> \$40

Step 5 - LLM processes result

Instructions

+

User request

+

Tool information/call

+

Tool result

|
LLM

One transaction stands out compared with the others: the \$1,800 transaction.

Here are your five transactions...

That is a basic agent loop.

10. Connect this to Context Engineering

Context engineering means deciding what information the model should receive.

```
+-----+
| Instructions |
+-----+
| Conversation history |
|   User          |
|   Assistant      |
|   User          |
|   Assistant      |
+-----+
| Tool definitions |
+-----+
| Tool calls/results |
+-----+
| RAG information  |
+-----+
|
| LLM
```

All of this must be managed properly. That is context engineering in practice.

11. A Mistake to Avoid

Suppose your tool returns an error:

Database connection failed.

The model should not respond with an invented balance such as “Your balance is \$25,000.” It does not have authoritative data.

Tool failure

|

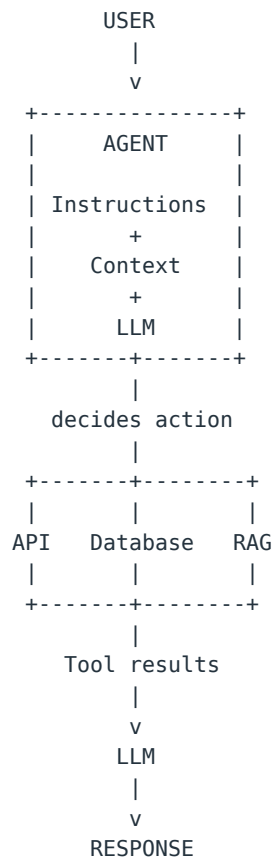
LLM / Application

|

Retry / fallback / explain failure

Production Agentic AI needs tool handling + error handling + guardrails + retries + validation, not merely good prompts.

12. Your Mental Model



This is the architecture the lesson is building toward.

13. What to Remember

Concept	Remember
User message	What the user wants.
System / developer instructions	How the model/application should behave.
Assistant message	Model-generated conversational content.
Tool definition	Tells the model what capability is available.
Tool call	Requests execution of that capability.
Tool result	Information returned from the external operation.

Core principle: Messages, tool information/results, conversation history, RAG information and instructions can all contribute to the context available to the LLM.

14. Quick Test

Q1. What is the difference between a user message and system/developer instructions?

Q2. Why should an LLM not simply answer “What is my current bank balance?” from its own model knowledge?

Q3. Explain the difference between tool definition -> tool call -> tool result.

Q4. Suppose `get_transactions()` fails with a database error. Should the LLM invent a likely transaction history? What should happen instead?

Q5. Walk through “Show me my last five transactions” from user request -> LLM -> tool -> external system -> tool result -> LLM -> final response.

Next lesson: Function / Tool Calling - turning the concepts above into an actual agent interaction.